Question 1

(a)(i)

Recall the 2D DFT definition

$$F(u,v) = \frac{1}{NN}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} f(x,y)e^{-j2\pi(ux/N+vy/N)}$$

$$G(u,v) = \frac{1}{NN}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} g(x,y)e^{-j2\pi(ux/N+vy/N)} = \frac{1}{NN}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} f(y,x)e^{-j2\pi(ux/N+vy/N)}$$

Relabel the variables: $m = y, n = x$

$$G(u,v) = \frac{1}{NN}\sum_{n=0}^{N-1}\sum_{m=0}^{N-1} f(m,n)e^{-j2\pi\left(\frac{un}{N}+\frac{vm}{N}\right)} = F(v,u)$$

[4]

(a)(ii)

$$G(u,v) = \frac{1}{NN}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} f(N-x,N-y)e^{-j2\pi(ux/N+vy/N)}$$

Relabel the variables: $m = N - x, n = N - y$

$$G(u,v) = \frac{1}{NN}\sum_{m=1}^{N}\sum_{n=1}^{N} f(m,n)e^{-j2\pi[\frac{u(N-m)}{N}+\frac{v(N-n)}{N}]} = \frac{1}{NN}\sum_{m=1}^{N}\sum_{n=1}^{N} f(m,n)e^{j2\pi\left(\frac{un}{N}+\frac{vm}{N}\right)}$$

The value of the above expression is the same as

$G(u,v) = \frac{1}{NN}\sum_{m=0}^{N-1}\sum_{n=0}^{N-1} f(m,n)e^{j2\pi\left(\frac{un}{N}+\frac{vm}{N}\right)}$ due to the assumed periodicity of the images involved.

Therefore,

$G(u,v) = F(-u,-v)$

[4]

(b)

Total energy: $E_{total} = \sum_{i=0}^{N-1}\lambda_i = \sum_{i=0}^{N-1} a^i = \frac{1-a^N}{1-a}$

Energy retained by first K components: $E_K = \sum_{i=0}^{K-1} a^i = \frac{1-a^K}{1-a}$

Fraction of energy retained:

$$\frac{E_K}{E_{total}} = \frac{1-a^K}{1-a^N}$$

We want:

$$\frac{1-a^K}{1-a^N} \geq 0.25$$

Solve for K:

$1 - a^K \geq 0.25(1 - a^N) \Rightarrow 1 - a^K \geq 0.25 - 0.25a^N \Rightarrow 0.75 - a^K \geq -0.25a^N \Rightarrow$

$a^K \leq 0.75 + 0.25a^N \Rightarrow K\log(a) \leq \log(0.75 + 0.25a^N) \Rightarrow K \geq \frac{\log(0.75 + 0.25a^N)}{\log(a)}$

Since $0 < a < 1$, $\log(a) < 1$, therefore when we multiply by $\frac{1}{\log(a)}$ the inequality changes direction.

If $N \to \infty$, $a^N \to 0$ therefore, $K \geq \frac{\log(0.75)}{\log(a)}$

[6]

(c)

The Discrete Cosine Transform (DCT) exhibits excellent energy compaction properties because it is equivalent to the real part of the Discrete Fourier Transform (DFT) applied to an even-symmetric extension of the original signal or image. This even-symmetric extension eliminates discontinuities at the boundaries, which are a major source of artificial high-frequency components in the standard DFT. As a result, the DCT represents the signal using only cosine basis functions, which align well with the structure of natural images that are typically smooth and dominated by low-frequency content. Consequently, the DCT concentrates most of the signal's energy into a few low-frequency coefficients, making it highly efficient for compression and sparse representation.

[6]

Question 2

(a)(i)

When two uncorrelated images are added, their histograms are convolved, which results in a broader intensity distribution in the sum image. This convolution increases the spread of pixel values, effectively widening the histogram and pushing more values toward the extremes of the intensity range. Since image contrast is directly related to the variability or spread of intensity values, the resulting image typically exhibits higher contrast than either of the original images. Thus, the sum of two uncorrelated images is expected to have greater contrast due to the increased dynamic range introduced by histogram convolution.

[4]

(a)(ii)

The image

$$g(x,y) = 4f(x,y) - f(x,y-1) - f(x,y+1) - f(x-1,y) - f(x+1,y)$$

is formed by taking a weighted sum of the central pixel and its neighbors, where the central pixel is multiplied by 3 and subtracted from three surrounding pixels. This operation enhances local intensity differences, acting like a high-pass or edge-emphasizing filter. As a result, the histogram of $g(x,y)$ will be wider and more centered around zero, with more values at both high and low extremes compared to the histogram of $f(x,y)$. While the histogram of $f(x,y)$ might be concentrated in a narrow range (especially in smooth images), the histogram of $g(x,y)$ reflects amplified contrast and increased variance due to the emphasis on spatial changes. Thus, we expect the histogram of $g(x,y)$ to be more spread out, possibly centered around zero, and with heavier tails.

[4]

(b)

Additive white noise is more noticeable in smooth or slowly varying regions of an image because these regions have low intrinsic variation, making any small fluctuations due to noise stand out clearly. In the spatial domain, smooth regions exhibit nearly constant or gradually changing pixel intensities, so even minor random perturbations introduced by noise disrupt the visual uniformity and become immediately perceptible. In contrast, regions with sharp edges or textures already contain significant local variation, masking the presence of noise.

In the frequency domain, white noise has equal power across all frequencies, including high frequencies. Smooth image regions correspond to low-frequency content, so when high-frequency noise is added, it introduces components that are not naturally present in the signal, making the noise more visually intrusive. On the other hand, edges and textures inherently contain high-frequency components, so the relative effect of white noise is less noticeable amid the already complex frequency structure.

Thus, both spatial and frequency perspectives explain why white noise disrupts smooth areas more severely.

[4]

(c)

To implement a bandpass filter in the spatial domain using convolution masks, one effective method is to subtract a low-pass filtered version of the image from a high-pass filtered version. This can be achieved by designing two convolution masks: one that performs low-pass filtering (e.g. a smoothing filter like a Gaussian or averaging kernel) and one that performs high-pass filtering (e.g. a Laplacian or a sharpening mask). The bandpass result is obtained by computing:

$$g(x,y) = h_{high}(x,y) * f(x,y) - h_{low}(x,y) * f(x,y)$$

where $*$ denotes convolution, $f(x,y)$ is the input image, $h_{low}(x,y)$ is the low-pass mask, and $h_{high}(x,y)$ is the high-pass mask. Alternatively, one can design a single bandpass mask by subtracting the low-pass mask from the high-pass mask directly and then convolving the image with the resulting bandpass mask.

This approach achieves bandpass behaviour because the low-pass filter suppresses high-frequency details and noise, while the high-pass filter suppresses the low-frequency background and smooth areas. Their difference isolates the intermediate frequencies — those not strongly passed by either filter alone — which correspond to edges, fine textures, and other mid-frequency structures. Therefore, this method selectively enhances features within a specific frequency range, effectively implementing bandpass filtering in the spatial domain.

[4]

(d)

The equivalent of the output of a Laplacian mask in 1D is the following:

$$y[n] = -x[n+1] + 2x[n] - x[n-1] = (x[n] - x[n+1]) - (x[n-1] - x[n])$$

I define the function $g[n] = x[n-1] - x[n]$. This is an approximation of the first derivative of $x[n]$.

We see that $y[n] = g[n+1] - g[n]$. This is an approximation of the first derivative of $g[n]$. Therefore, $y[n]$ is an approximation of the second derivative of $x[n]$.

The extension to 2D is straightforward.

[4]

Question 3

(a)

Using standard DFT properties, the 2D DFT

$$H(u,v) = \sum_{-1}^{1}\sum_{-1}^{1} h(x,y)e^{-j2\pi\left(\frac{xu+yv}{N}\right)}$$

$$= \frac{1}{8}[4 + e^{-j2\pi\frac{v}{N}} + e^{j2\pi\frac{v}{N}} + e^{-j2\pi\frac{u}{N}} + e^{j2\pi\frac{u}{N}}]$$

$$= \frac{1}{8}[4 + e^{-j2\pi\frac{v}{N}} + e^{j2\pi\frac{v}{N}} + e^{-j2\pi\frac{u}{N}} + e^{j2\pi\frac{u}{N}}] = \frac{1}{8}[4 + 2\cos(2\pi\frac{u}{N}) + 2\cos\left(2\pi\frac{v}{N}\right)]$$

In the above relationship we can easily see that there are frequency pairs for which $H(u,v) = 0$, as for example $u = N/2$ and $v = N/2$. In that case $4 + 2\cos\left(2\pi\frac{N}{2}\right) + 2\cos\left(2\pi\frac{N}{2}\right) = 4 + 2\cos(\pi) + 2\cos(\pi) = 0$. Therefore, Pseudo Inverse Filter must be used.

[4]

(b)

The DFT of the Laplacian operator is given below:

$$C(u,v) = \sum_{-1}^{1}\sum_{-1}^{1} c(x,y)e^{-j2\pi\left(\frac{xu+yv}{N}\right)}$$

$$= c(0,0) + c(0,1)e^{-j2\pi\frac{v}{N}} + c(0,-1)e^{j2\pi\frac{v}{N}} + c(1,0)e^{-j2\pi\frac{u}{N}} + c(-1,0)e^{j2\pi\frac{u}{N}}$$

$$= 4 - e^{-j2\pi\frac{v}{N}} - e^{j2\pi\frac{v}{N}} - e^{-j2\pi\frac{u}{N}} - e^{j2\pi\frac{u}{N}} = 4 - 2\cos\left(2\pi\frac{u}{N}\right) - 2\cos\left(2\pi\frac{v}{N}\right)$$

We know that $0 \le u, v < N - 1$.

The only frequency pair for which $C(u,v) = 0$ is the pair $(u,v) = (0,0)$. However, for $(u,v) = (0,0)$ the term $|H(u,v)|^2$ must be not zero. This is always the case because $H(u,v)$ is a lowpass filter. Therefore, there aren't any frequency pairs for which the original image cannot be recovered using equation above.

[4]

(c)(i)

The frequency-domain Wiener filter is

$$W(u,v) = \frac{H(u,v)^*}{|H(u,v)|^2 + \frac{S_n(u,v)}{S_f(u,v)}}$$

Therefore, $\frac{S_n(u,v)}{S_f(u,v)} = \frac{1}{K}$ or $\frac{S_f(u,v)}{S_n(u,v)} = K$.

[4]

(c)(ii)

When K is very large (high SNR) then

$W(u,v) = \frac{H(u,v)^*}{|H(u,v)|^2} = \frac{1}{H(u,v)}$ (Inverse Filtering)

In that case the filter assumes that there is no noise at all.

When $K \to 0$ then $W(u,v) \to 0$

In that case the Wiener filter becomes overly conservative, suppressing both noise and image details. It essentially acts like a null filter, outputting near-zero values to avoid introducing error, but at the cost of completely losing image content.

[4]

c(iii)

To estimate the parameter K in Wiener filtering, one practical approach is to compare the average noise level to the average signal strength in the degraded image. This can be done by identifying a relatively flat area of the image—such as a uniform background—where any variation is likely due to noise, and measuring how much the pixel values fluctuate there. At the same time, a representative region of the image that contains useful detail is used to estimate the overall signal variation. The ratio between these two estimates gives a value for K, helping to balance how much the filter suppresses noise versus how much image detail it preserves. This method is grounded in statistical reasoning and adapts the filter to the actual conditions of the image.

[4]

Question 4

(a)(i)

$M = 1$ **(single pixel blocks)**

Huffman Coding

We apply Huffman coding to the symbols A, B, C with probabilities:

- A: 0.10
- B: 0.20
- C: 0.70

Generated Huffman Codes

- C → 1
- B → 01
- A → 00

Entropy calculation

$H = -\sum p_i \log_2(p_i) = -[0.1\log_2(0.1) + 0.2\log_2(0.2) + 0.7\log_2(0.7)] \approx 1.157$ bits/symbol

Average codeword length

$L_{avg} = 0.1 \cdot 2 + 0.2 \cdot 2 + 0.7 \cdot 1 = 1.3$ bits/symbol

[2]

(a)(ii)

$M = 2$ **(blocks of 2 pixels)**

There are 9 possible symbol pairs: AA, AB, AC, BA, BB, BC, CA, CB, CC.

Joint Probabilities

Assuming independence:

AA $0.10 \times 0.10 = 0.01$

AB $0.10 \times 0.20 = 0.02$

AC $0.10 \times 0.70 = 0.07$

BA $0.20 \times 0.10 = 0.02$

BB $0.20 \times 0.20 = 0.04$

BC $0.20 \times 0.70 = 0.14$

CA $0.70 \times 0.10 = 0.07$

CB $0.70 \times 0.20 = 0.14$

CC $0.70 \times 0.70 = 0.49$

Huffman Codes for Blocks

Sorted by probability (highest first), the codes assigned are:

Block Symbol	Probability	Huffman Code
CC	0.49	0
CB	0.14	101
BC	0.14	110
AC	0.07	1111
CA	0.07	1110
BB	0.04	1000
AB	0.02	10010
BA	0.02	100111
AA	0.01	100110

Entropy calculation

$H = -\sum p_i \log_2(p_i) = -[0.1\log_2(0.1) + 0.2\log_2(0.2) + 0.7\log_2(0.7)] \approx 1.157$ bits/symbol

Average codeword length

$L_{avg} = 1.165$ bits/symbol

[3]

(a)(iii)

$M = 1$

Efficiency$= \frac{H}{L_{avg}} \approx 0.890$

Redundancy$= 1 -$Efficiency≈ 0.11

[2]

$M = 2$

Efficiency$= \frac{H}{L_{avg}} \approx 0.993$

Redundancy$= 1 -$Efficiency≈ 0.007

[2]

(b)(i)

Huffman Coding

We apply Huffman coding to the symbols A, B, C with probabilities:

- C: 0.125

- A: 0.125
- T: 0.25
- S: 0.5

Generated Huffman Codes

- C → 111
- A → 110
- T → 10
- S → 0

[3]

(b)(ii)

Entropy calculation

$H = 1.75$ bits/symbol

Average codeword length

$L_{avg} = 1.75$ bits/symbol

Efficiency$= \frac{H}{L_{avg}} = 1$

Redundancy$= 0$

The redundancy is 0 because the probabilities are negative powers of 2.

[2]

(b)(iii)

We now encode **"CAT"** using this code:

- C → 111
- A → 110
- T → 10

Concatenated bitstream:

111 110 10

(8 bits total)

Bit #	Value	Error Probability
1	1	0.10
2	1	0.10
3	1	0.10
4	1	0.10
5	1	0.10
6	0	0.01

Bit #	Value	Error Probability
7	1	0.10
8	0	0.01

Probability that all 8 bits are correctly received

We multiply the probabilities of no error for each bit:

$$P(\text{correct}) = (1 - 0.10)^6 \cdot (1 - 0.01)^2 = (0.9)^6 \cdot (0.99)^2 = 0.531441 \cdot 0.9801 \approx 0.521$$

Final Answer

$$P(\text{error in CAT}) = 1 - 0.521 = 0.479$$

That is, the probability that the word **"CAT"** is received incorrectly (i.e., **at least one bit is wrong**) is approximately 47.9%.

[6]